

Lab. 1
Environment Setup & Introduction to RDBMS

Contents

1) Course Syllabus
2) What is a Database? (and why we use one)
3) Setting Up & Running JavaScript in VS Code
4) JavaScript Basics: Variables, Types, Decisions, Loops
5) Functions & Introduction to Node.js
6) Data: Objects, Arrays & a Temporary Database
7) Where We're Going Next
8) Summary

Course Syllabus

- **Lecturer / Organizer**

  Name: Assoc. Prof. Dr. Somchart Fugkeaw
  Contact: somchartf@yahoo.com

- **Instructor**

  Name: Dr. Suparchoek Wangmanaopituk
  Contact: suparchoek.aj@siit.tu.ac.th

Course Syllabus (Contd.)

- **Course Description**
  - Practical usage of core database manipulation functions with SQL
  - Designing relational schemas and database-backed applications
  - Building a full-stack CRUD application on a single web-based stack using MySQL and Node.js
  - Using transactions, stored procedures, triggers and security features for robust applications

Course Syllabus (Contd.)

- **Class time**
  - *Section 5:* Thursday 9:00-12:00
  - *Section 6:* Thursday 13:00-16:00

- **Classroom**
  - Location : 2301 IT Lab, 3thfloor of the clock tower building, Bangkadi campus campus.

- **Textbook**
  - Database System Concepts: Abraham Silberschatz, Henry Korth and S. Sudarshan, 7th Edition, McGraw-Hill Education

Course Syllabus (Contd.)

- **Lab sheets**
  - Available online on Google Classroom. Join using your SIIT student account.
  - Each sheet explains the lab in detail; the slides summarize it and list the in-class exercises.
  - Your TA grades your work based on the Lab assignment instructions.
- **Course Schedule**

** Refer the course syllabus*

Course Syllabus (Contd.)

➢ Course Assessment

Criteria	Percentage (%)	Remark
Attendance	10	GC attendance Check (with selfie upload; within 30 minutes of class start) & Check at the end of the class by exercise checking
Lab Assignment	15	Submit at the end or week after the class (TA's approval)
Quiz	5	Pop quiz
Project Proposal	10	
Final Project	30	
Final Exam	30	
Total	100	

Course Syllabus (Contd.)

➢ Grading Policy

- Scores are curved. Your final grade depends on the distribution of total class scores. All sections are curved and graded together.

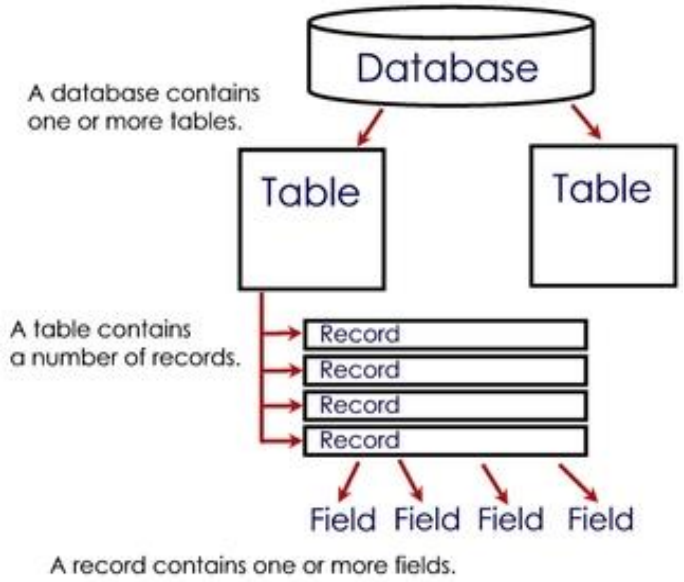
➢ Absent Policy

– Absent students MUST submit their lab assignment with the TA's acknowledgement in the next class.

– With a proper missing-excuse document: full score. Without: only 70% of the full score.

What is a Database?

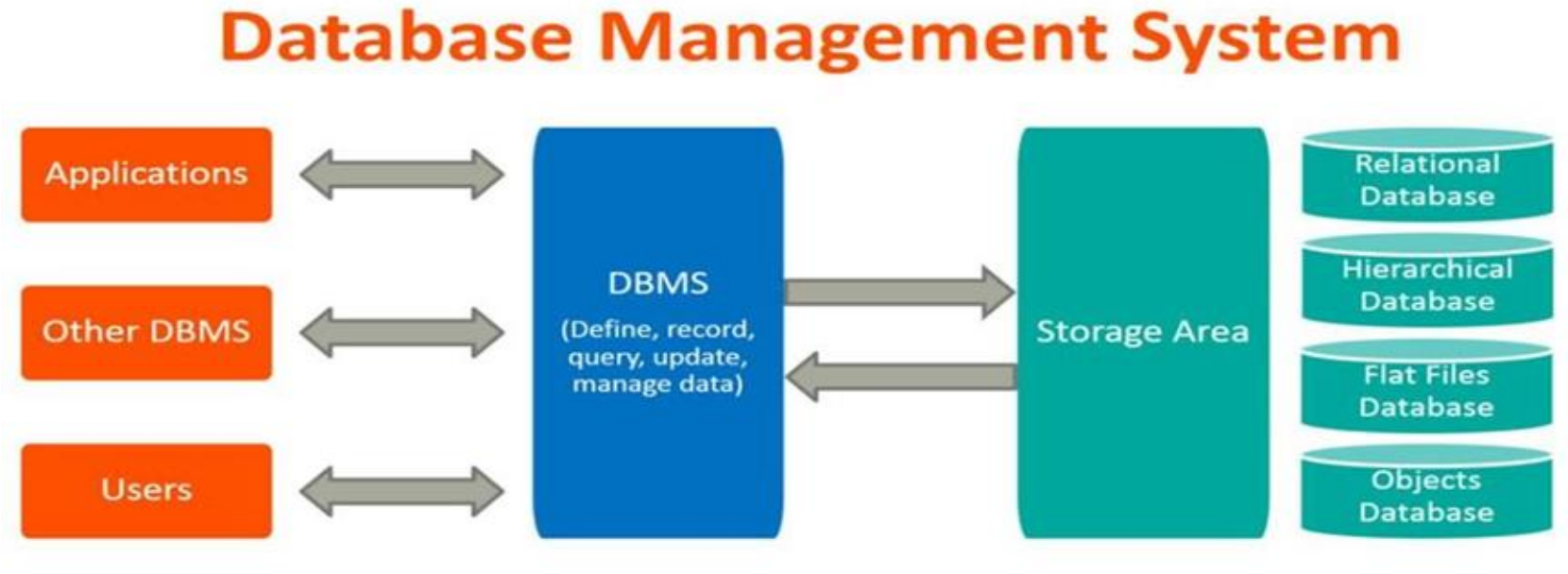
- A database is an organized collection of related data — for example, all the students in a school, or all the products in an online shop.
- A DBMS (Database Management System) is the software that lets us add, find, change and remove that data safely



ID	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

What is a Database? Contd.

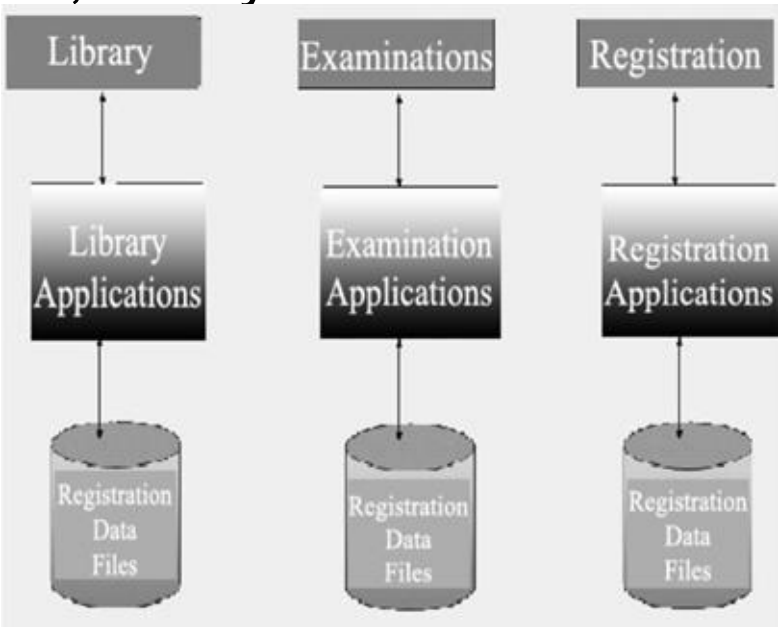
- You already use databases every day: contacts on your phone, a library catalogue, your bank's records.
- We will use MySQL, a popular, free relational database that stores data in tables (rows and columns).



Why a Database? (instead of a single file)

- Could we just keep data in a notebook or one big text file? For anything real, that breaks down:
  - Hard to search quickly as data grows.
  - Easy to duplicate or mistype information (inconsistency).
  - Difficult for many people to use at the same time, safely.

- A database solves these: fast search, consistency, sharing, and reliability.
- In this course we build a small web application backed by a MySQL database.



Setting Up & Running JavaScript in VS Code

➤ **One simple, consistent stack; you learn one toolchain from start to finish:**

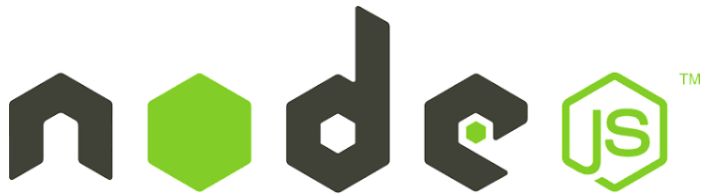
- MySQL — the database that stores our data.
- JavaScript — the programming language we write everything in.
- Node.js — lets us run JavaScript on our computer / server (talks to the database).

*On macOS use the native builds; on Windows-in-Parallels use the ARM64 builds.

* Install Vscode as a code editor as well.



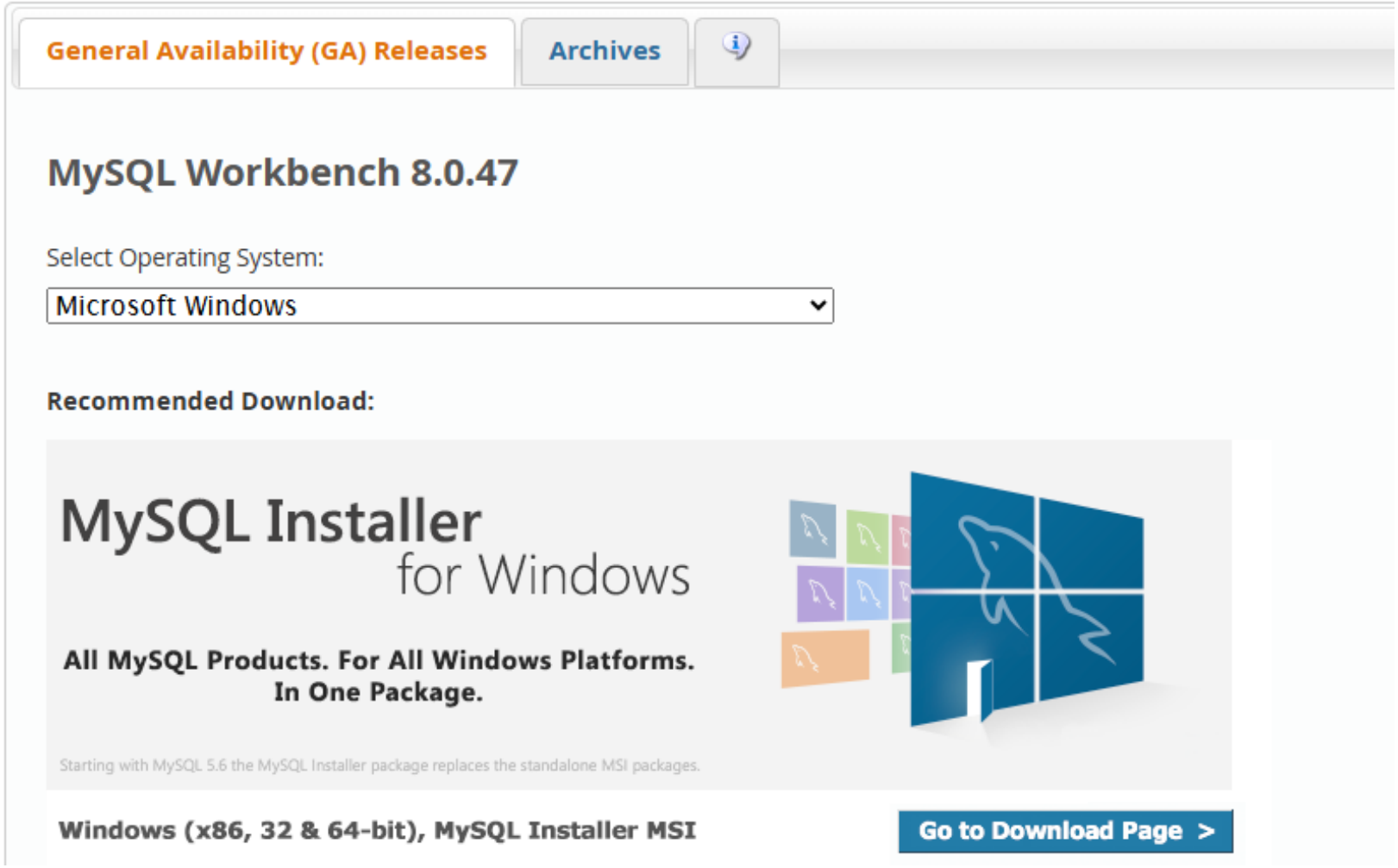




Setting Up & Running JavaScript in VS Code (Contd.)

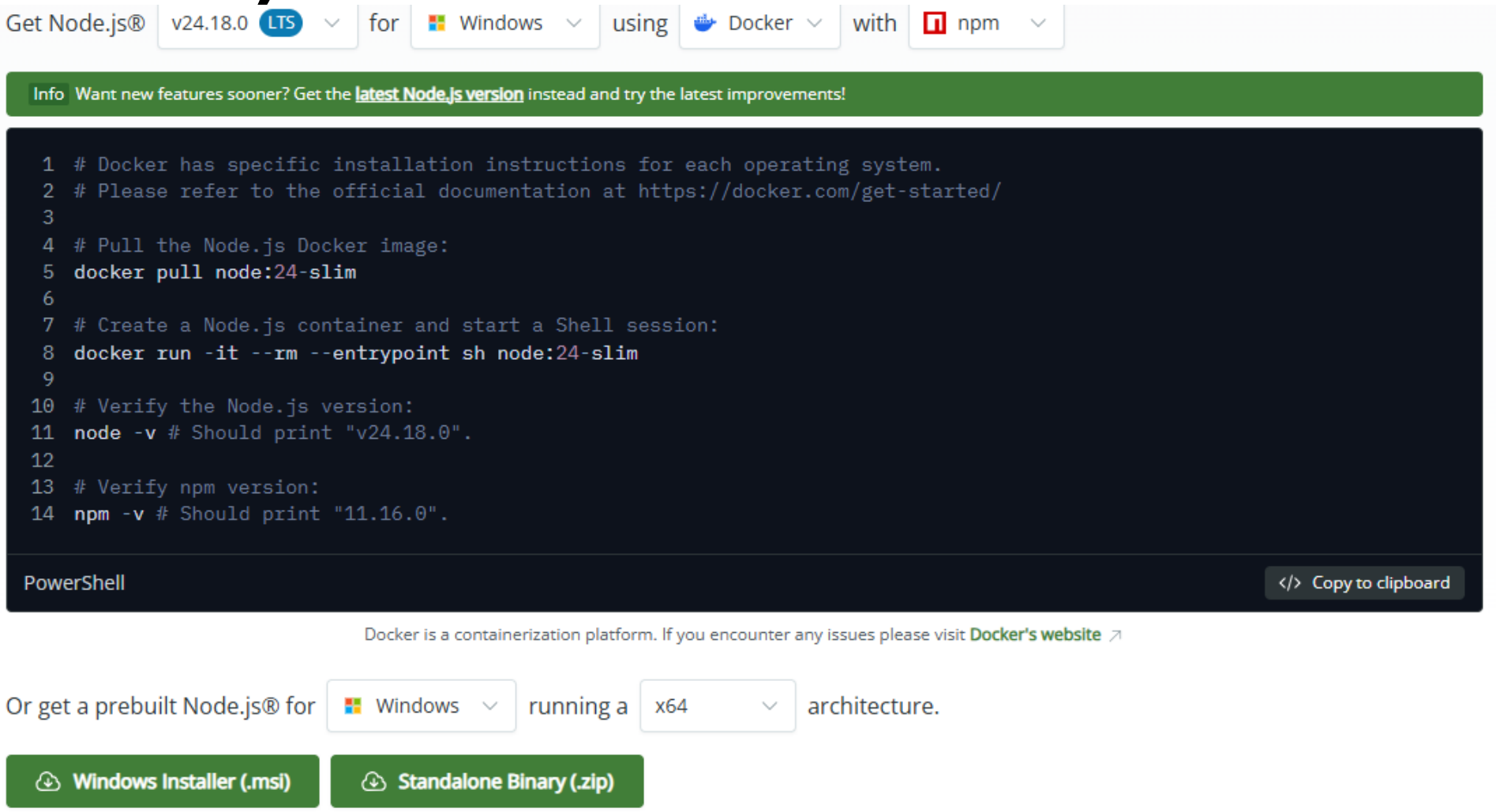
Download & install MySQL according to your OS
https://dev.mysql.com/downloads/workbench/

*** Make sure to install the Full version



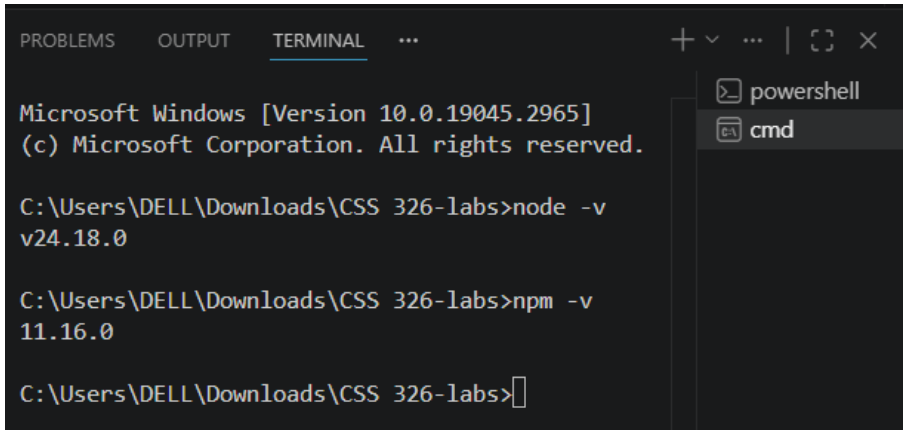
Setting Up & Running JavaScript in VS Code (Contd.)

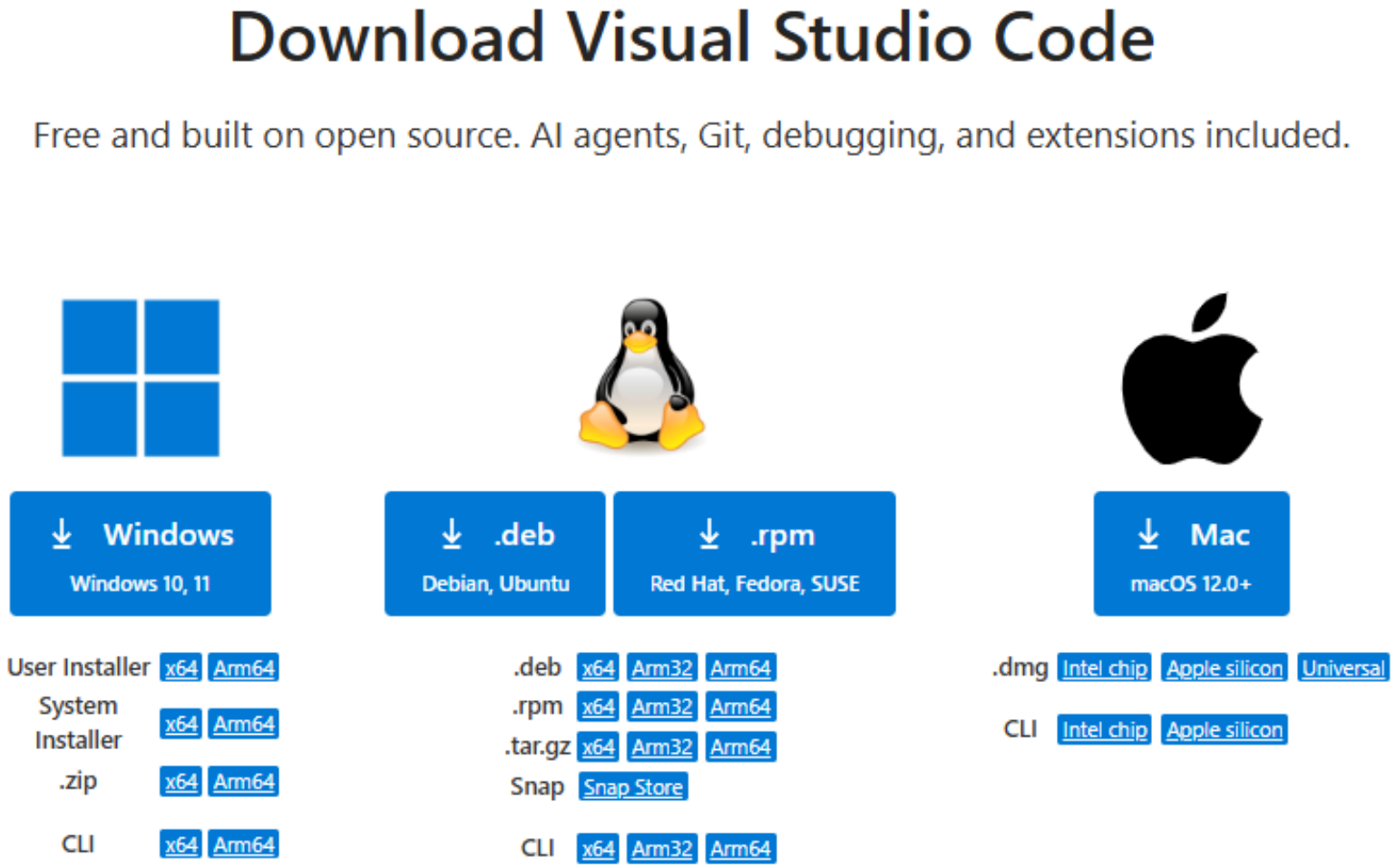
Download & install node JS according to your OS https://nodejs.org/en/download



Setting Up & Running JavaScript in VS Code (Contd.)

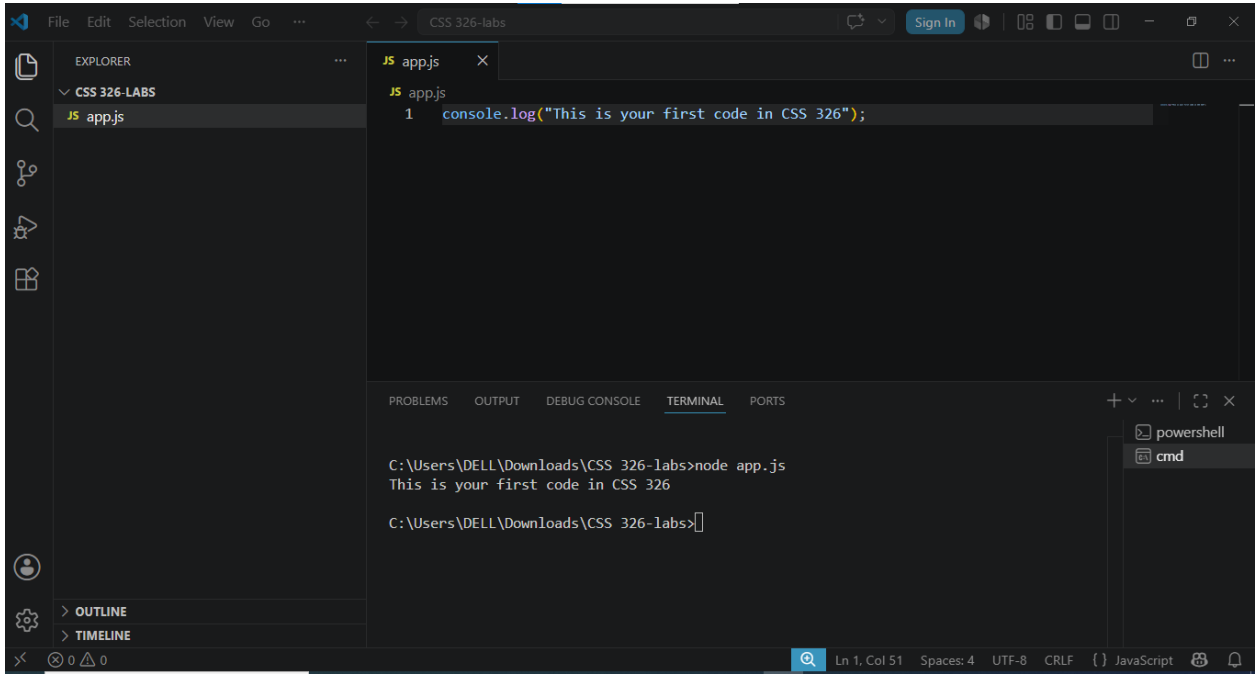
Download & install VS code according to your OS & check the versions of node and npm https://code.visualstudio.com/download?_exp_download=fb315fc982





Setting Up & Running JavaScript in VS Code (Contd.)

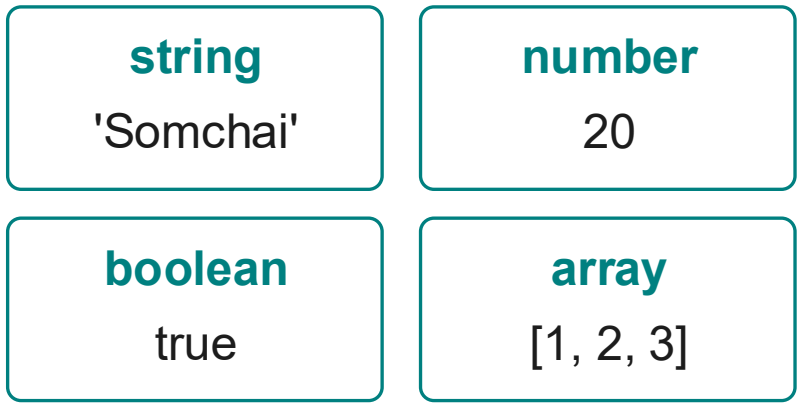
1) Open a folder (CSS 326 Lab 1 or other) in VS Code
2) Create app.js and type your code
3) Open Terminal (View › Terminal) and run; node app.js



JavaScript Basics: Variables, Types, Decisions, Loops

A variable is a named box that holds a value; every value has a type.

- **Create variables with let or const:**
  - let age = 20;
  - const pi = 3.14;
- **Main types you'll use:**
  - number — 20, 3.14
  - string — text in quotes
  - boolean — true / false
- console.log(...) prints a value.



JavaScript Basics: Variables, Types, Decisions, Loops (Contd.)

Lab Exercise 1: **Variables, types & running in VS Code**

- In app.js, declare name (string), age (number) and isStudent (boolean).
- Print a sentence such as: Anong is 19 (student: true).
- Run it with node app.js in the VS Code terminal.

Starter code

```
let name = '____';
let age = ____;
let isStudent = ____;
console.log(name + ' is ' + age + ' (student: ' + isStudent + ')');
```

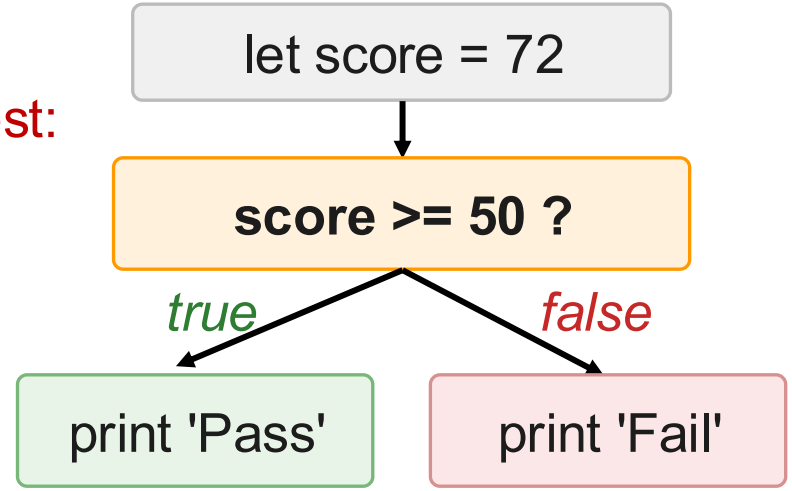
JavaScript Basics: Variables, Types, Decisions, Loops (Contd.)

Making Decisions: if / else

Programs choose what to do based on a condition that is true or false.

- **Comparisons give true / false:**
  - `>` `<` `>=` `<=` `===` `!==`
- if runs a block when the condition is true; else covers the rest:

```
if (score >= 50) {
  console.log('Pass');
}
else {
console.log('Fail');
}
```



JavaScript Basics: Variables, Types, Decisions, Loops (Contd.)

Lab Exercise 2: **Conditionals — grade classifier**

- Store a score (0–100) in a variable.
- Use if / else if / else to print: A (>=80), B (>=70), C (>=50), else F.
- Test it by changing the score and re-running.

Starter code

```
let score = 72;
if (score >= 80) {
  console.log('A');
} else if (score >= 70) {
  // ... continue the chain ...
}
```

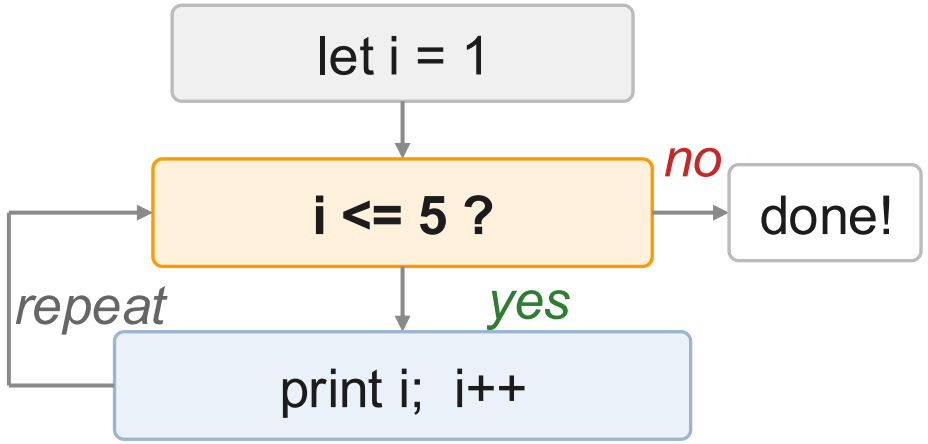
JavaScript Basics: Variables, Types, Decisions, Loops (Contd.)

Repeating Work: Loops

A loop repeats a block of code while a condition stays true.

- **A for loop counts for us:**

```
for (let i = 1; i <= 5; i++) {
 console.log(i);
}
```



- **Three parts: start, condition, step.**
- **A while loop repeats until its condition is false.**

JavaScript Basics: Variables, Types, Decisions, Loops (Contd.)

Lab Exercise 3: Loops & a simple stopwatch

- Part A: use a for loop to print the numbers 1 to 10.
- Part B: build a countdown that prints 5, 4, 3, 2, 1 then 'Time's up!'.
- Hint: setInterval(fn, 1000) runs fn every second; clearInterval stops it.

Starter code (Part B)

```
let t = 5;
const timer = setInterval(() => {
  console.log(t);
  t--;
  if (t < 0) { clearInterval(timer); console.log("Time's up!"); }
}, 1000);
```

Functions & Introduction to Node.js

Functions: Reusable Blocks of Code

- A function is a named block you can run again and again, optionally with inputs (parameters).
  - function add(a, b) { return a + b; }
  - console.log(add(2, 3)); // 5
- Functions let us avoid repeating ourselves and keep programs organized.
- Soon, each database operation (add / find / remove a record) will be its own function.

Functions & Introduction to Node.js (Contd.)

Lab Exercise 4: Type errors — predict, then catch

- Predict whether the line below will crash, then run it to check .toUpperCase() only exists on strings, so calling it on a number throws a TypeError.
- Wrap the risky line in try { ... } catch (err) { ... } so the error is caught and the program keeps running instead of crashing..

Starter code (Part B)

```
// 1) predict: does this line crash? run it to check
let price = 25;
console.log(price.toUpperCase());   // price is a number!
// 2) wrap the risky line in try/catch
try {
  console.log(price.toUpperCase());
} catch (err) {
  console.log('Type error:', err.message);
}
```

Functions & Introduction to Node.js (Contd.)

Introduction to Node.js

- You've been running JavaScript with Node.js already. So, what is it?
- Normally JavaScript runs inside a web browser; Node.js lets us run it directly on our computer.
- **Why this matters for databases:**
  - A web app's back-end (the part that talks to the database) runs on a server.
  - Node.js is what runs that back-end code - the bridge to MySQL.
  - npm (installed with Node.js) adds libraries - later we add mysql2.

Data: Objects, Arrays & a Temporary Database

- An object groups related values together; think of it as one record (one row):

```
let student = { id: 1, name: 'Somchai', age: 20 };
```

- An array holds many items in a list — think of it as a table of records:

```
let students = [ student1, student2, student3 ];
```

- Access values with a dot or an index: student.name and students[0]
- Same shape in every row (id, name, age) = the idea behind a table's schema.

Data: Objects, Arrays & a Temporary Database (Contd.)

A Temporary In-Memory Database

An array + a function per operation. Save as students.js and run; node students.js

```
let students = [];                    // our 'table'

function addStudent(id, name, age) {     // like INSERT
  students.push({ id, name, age });
}
function findById(id) {                  // like SELECT ... WHERE
  return students.find(s => s.id === id);
}

addStudent(1, 'Somchai', 20);
addStudent(2, 'Suda', 21);
console.log(students);        // all records
console.log(findById(2));     // one record
```

In memory only — data disappears when the program ends. MySQL makes it permanent (Lab 4).

Data: Objects, Arrays & a Temporary Database (Contd.)

Lab Exercise 5: Objects, arrays & the in-memory database

- Start from the students.js example.
- Add listAll() to print every student, and removeById(id) using filter.
- Add three students, remove one, then call listAll().

Starter code (Part B)

```
function listAll() {
  console.log(students);
}
function removeById(id) {
  students = students.filter(s => s.id !== id);
}
```

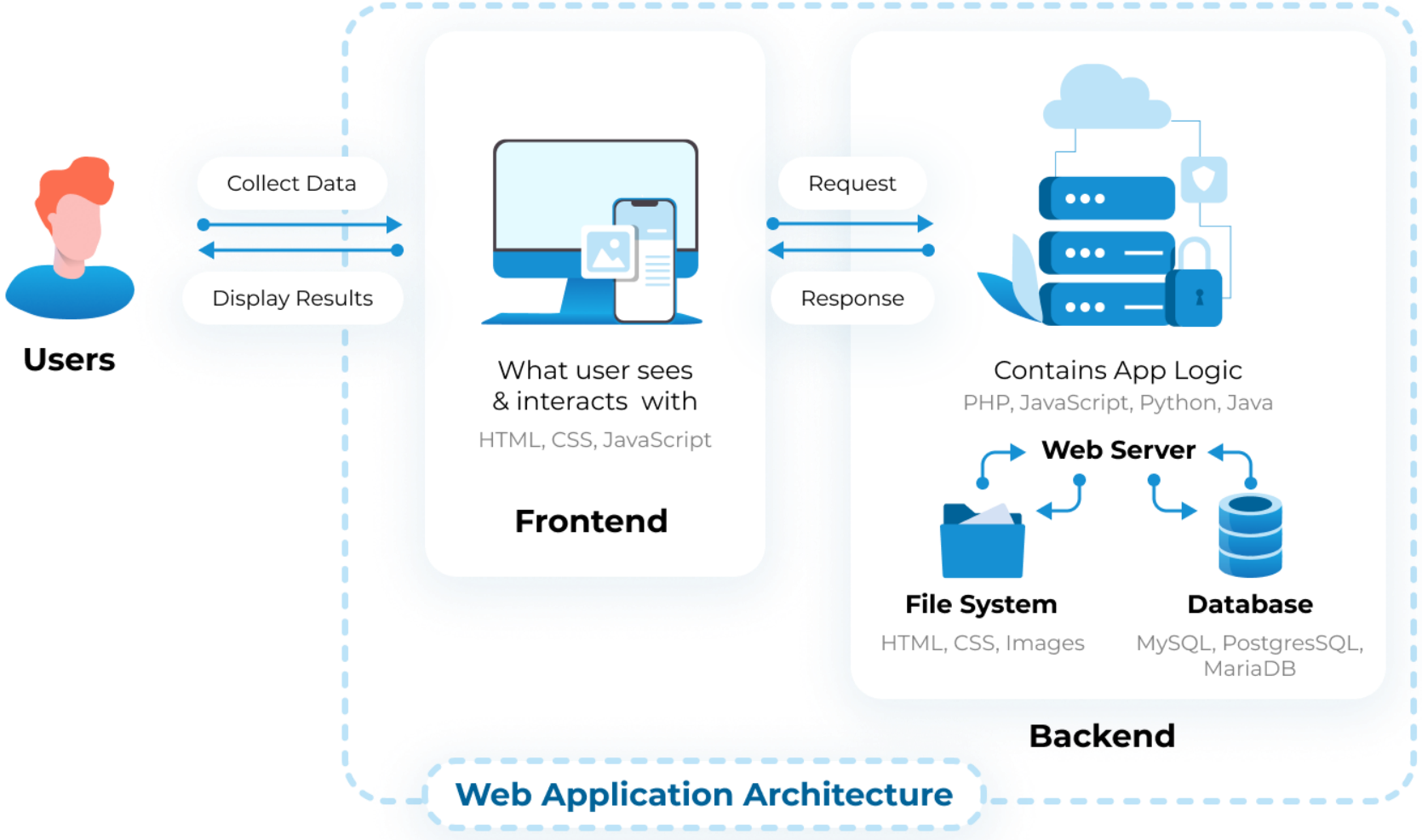
Where We're Going Next

- Today's data was temporary. From here we make it permanent with a real database.
- Lab 2: design the database — tables, columns and relationships (schema).
- Lab 3: write real SQL — INSERT, SELECT, UPDATE, DELETE in MySQL.
- Lab 4: connect this Node.js code to MySQL (mysql2) — the array becomes a real, persistent database.
- Same four ideas (add / find / change / remove) — just stored safely on disk.

Summary

- A database is organized, shared, reliable data; a DBMS (MySQL) manages it.
- JavaScript basics: variables & types, decisions (if/else), loops, and functions.
- Write your code in VS Code and run it with node app.js.
- Objects are records, arrays are tables; an in-memory database is temporary; MySQL makes it permanent (Lab 4).
- **Complete Exercises 1–5 in class (Save the codes and output in a word file), then submit Lab Exercise #1.**

Additional: Basic Components of Web Applications



Additional: Basic Components of Web Applications (Contd.)

Few of the popular DBMS

Rank Jul 2026	Rank Jun 2026	Rank Jul 2025	DBMS	Database Model	Score Jul 2026	Score Jun 2026	Score Jul 2025
1.	1.	1.	Oracle	Relational, Multi-model	1132.37	-7.67	-84.69
2.	2.	2.	MySQL	Relational, Multi-model	846.46	-9.83	-94.27
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	698.96	+0.92	-72.18
4.	4.	4.	PostgreSQL	Relational, Multi-model	687.80	-0.43	+6.92
5.	5.	5.	MongoDB	Document, Multi-model	386.62	-1.36	-17.22
6.	6.	6.	Snowflake	Relational	216.06	+1.49	+39.89
7.	7.	↑ 12.	Databricks	Multi-model	164.08	+6.50	+56.04
8.	8.	↓ 7.	Redis	Key-value, Multi-model	153.81	+3.79	+4.09
9.	9.	↓ 8.	IBM Db2	Relational, Multi-model	112.99	-0.54	-14.52
10.	10.	↑ 11.	Apache Cassandra	Wide column, Multi-model	103.06	+0.08	-5.70

Thank you…